

ICS 604

Applied Data Science

Department of Information and Computer Sciences
University of Hawai`i at Mānoa

Kyungim Baek

Lecture 21

- Non-Linear Regression
 - Polynomial regression (cont'd.)
- Introduction to Time Series Analysis
 - Time series data
 - Pandas and time series
 - Modeling the trend and seasonality

Previously... Polynomial regression

- Rather than model the data with a first-degree polynomial, we will use a higher-degree polynomial.
- Two-step process:
 1. Transform x into higher degree features.
 2. Fit the linear model using the transformed polynomial features.
- For example,

$$y = \beta_0 \cdot 1 + \beta_1 \cdot A + \beta_2 \cdot B + \beta_3 \cdot C$$

such that $A = x$, $B = x^2$ and $C = x^3$

```
x = np.array([1, 2, 3, 4, 5])  
x.reshape(-1, 1)
```

```
array([[1],  
       [2],  
       [3],  
       [4],  
       [5]])
```

```
from sklearn.preprocessing import PolynomialFeatures
```

```
x = np.array([1, 2, 3, 4, 5])
```

```
poly = PolynomialFeatures(degree=3)  
poly.fit_transform(x.reshape(-1, 1))
```

```
array([[ 1.,  1.,  1.,  1.],  
       [ 1.,  2.,  4.,  8.],  
       [ 1.,  3.,  9., 27.],  
       [ 1.,  4., 16., 64.],  
       [ 1.,  5., 25., 125.]])
```

$x_1, x_2 \leftarrow$ two predictors

cost: 1

Linear: x_1, x_2

quadratic: x_1^2, x_2^2

cubic: x_1^3, x_2^3

interaction: $x_1 x_2$

mixed cubic: $x_1 x_2^2, x_1^2 x_2$

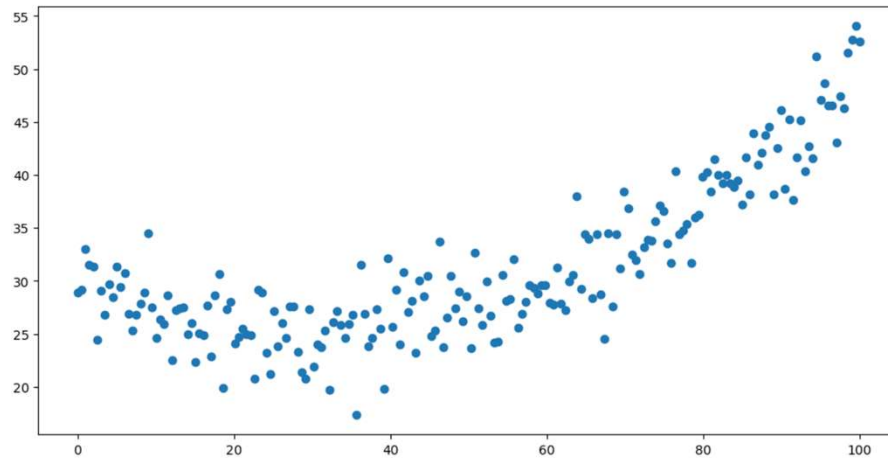
calculate the # of output features

fit() transform()

transformed data matrix

```
errors = np.random.normal(0, 3, size=200)
x = np.linspace(0, 100, 200)
y = 30 + (-0.3 * x) + (0.005 * x ** 2) + errors
```

```
plt.figure(figsize=(12, 6))
plt.scatter(x, y)
```



```
# we take a small subsample of x
np.random.seed(46)
```

```
subset_indices = np.random.choice(np.arange(len(x)), size=20)
subset_indices.sort()
subset_indices
```

```
array([ 22,  30,  31,  32,  40,  55,  59,  60,  69,  72,  79,  93,  98,
        116, 130, 146, 147, 155, 186, 189])
```

```
x[subset_indices]
```

```
array([11.05527638, 15.07537688, 15.57788945, 16.08040201, 20.10050251,
        27.63819095, 29.64824121, 30.15075377, 34.67336683, 36.18090452,
        39.69849246, 46.73366834, 49.24623116, 58.29145729, 65.32663317,
        73.36683417, 73.86934673, 77.88944724, 93.46733668, 94.97487437])
```

```
temp = x[subset_indices].reshape(-1, 1)
print(temp.shape)
temp[0:5]
```

```
(20, 1)
array([[11.05527638],
       [15.07537688],
       [15.57788945],
       [16.08040201],
       [20.10050251]])
```

```
poly = PolynomialFeatures(degree=2)
X_vals_transformed = poly.fit_transform(x[subset_indices].reshape(-1, 1))
X_vals_transformed[0:5]
```

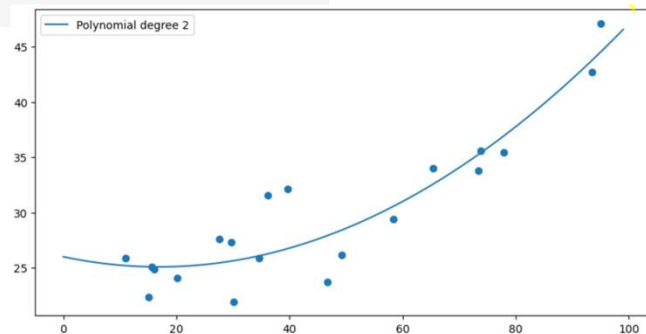
```
array([[ 1.      , 11.05527638, 122.21913588],
       [ 1.      , 15.07537688, 227.26698821],
       [ 1.      , 15.57788945, 242.67063963],
       [ 1.      , 16.08040201, 258.5793288 ],
       [ 1.      , 20.10050251, 404.03020126]])
```

$$y = \beta_0 + \beta_1 x + \beta_2 x^2$$

```
from sklearn.linear_model import LinearRegression

lin_reg = LinearRegression()
lin_reg.fit(X_vals_transformed, y[subset_indices].reshape(-1, 1))
x_axis = np.arange(0, max(x)).reshape(-1, 1)
X_axis_transformed = poly.transform(x_axis)
y_hat = lin_reg.predict(X_axis_transformed)

plt.figure(figsize=(10, 5))
plt.scatter(x[subset_indices], y[subset_indices])
plt.plot(x_axis, y_hat, label="Polynomial degree %s" % 2)
plt.legend();
```



Increasing the polynomial degree

- We said earlier that the best model is the one that minimizes the RSS.
- In the following example, we see that we can improve the fit by choosing a higher-degree polynomial.

```
plt.figure(figsize=(15, 10))

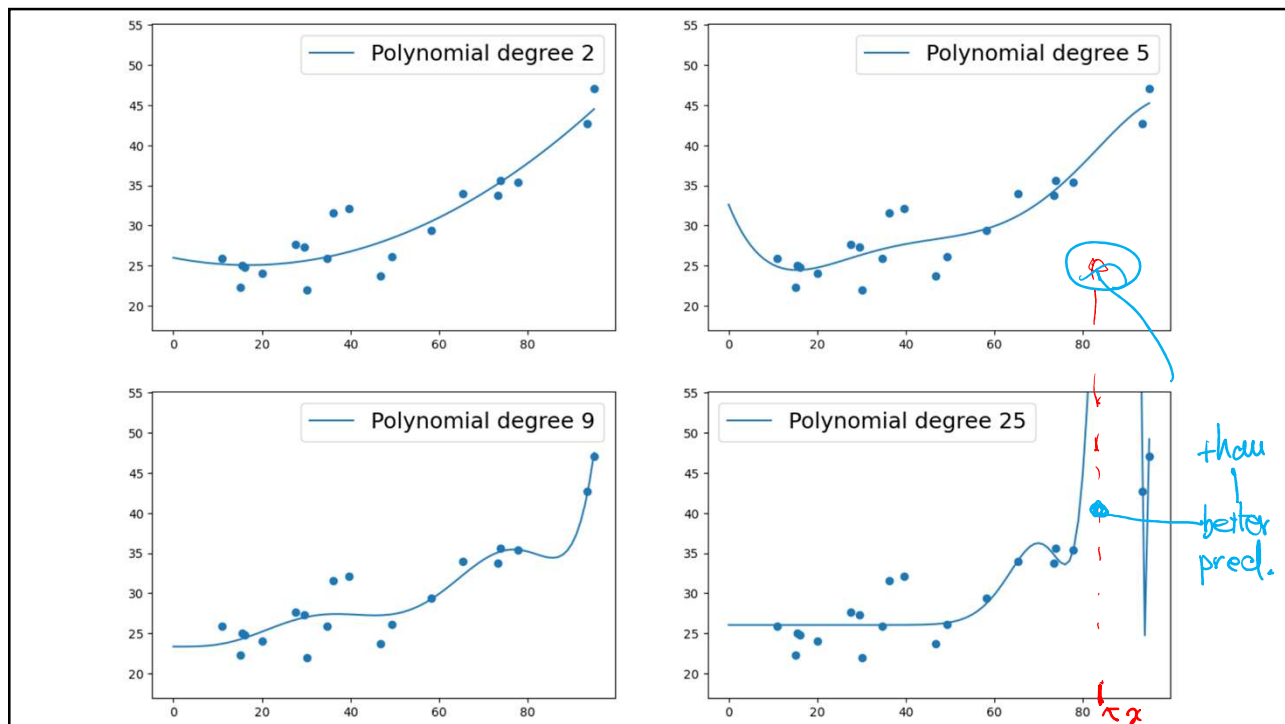
x_axis = np.arange(0, max(x)).reshape(-1, 1)

for i, polDegree in enumerate([2, 5, 9, 25]):
    plt.subplot(2, 2, i+1)

    plt.scatter(x[subset_indices], y[subset_indices])
    poly = PolynomialFeatures(degree=polDegree)
    X_vals_transformed = poly.fit_transform(x[subset_indices].reshape(-1, 1))
    X_axis_transformed = poly.transform(x_axis)

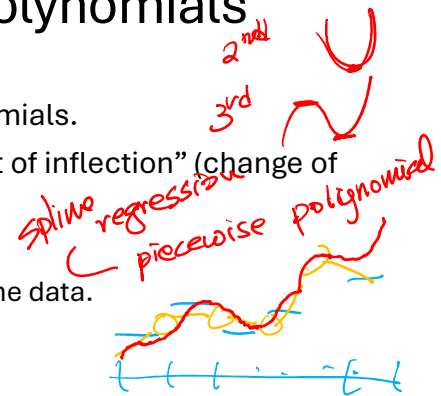
    lin_reg = LinearRegression()
    lin_reg.fit(X_vals_transformed, y[subset_indices].reshape(-1, 1))
    y_hat = lin_reg.predict(X_axis_transformed)

    plt.plot(x_axis[0:-4], y_hat[0:-4], label="Polynomial degree %s" % polDegree)
    plt.ylim(min(y[subset_indices]) - 5, max(y[subset_indices]) + 8)
    plt.legend(fontsize=18)
```



Shortcomings of higher-order polynomials

- Better fits are often achieved by higher-order polynomials.
- An n^{th} degree polynomial will have up to $n - 1$ “point of inflection” (change of direction point).
- Overfits the data
 - Oscillations are very unlikely to be characteristic of the data.
 - The model is capturing the noise in the data.
- This is an example of overfitting.
 - Even though this model passes through most of the data, it will fail to generalize on unseen data.
- We can use a train/validation split to select the best polynomial degree.
 - This strategy is widely applicable across machine learning and statistical learning.



Time Series Analysis

Introduction to time series analysis

- We will cover a few essential concepts for analyzing or modeling time series data.
- This is meant to be an introductory overview.
 - Time series are complex and warrant an entire course.
 - Our goal is to develop intuition and introduce key concepts rather than provide a comprehensive treatment.
- We will only consider univariate time series.

What are time series?

- Time series are data where the variation of some quantity is followed over time.
 - The temporal dimension is central, capturing patterns such as growth, cycles, and fluctuations.
- Examples:
 - The number of cars that pass an intersection over time.
 - The number of travelers arriving at HNL over time.
 - Temperature fluctuations over time.

Time series data

- Time series data is different from data we used so far with regards to two aspects.
 1. Data points are not independent of each other.
 - Observation at time t is dependent on observation at time $t - 1$.
 - Therefore, ordering of the data matters.
 2. Data points are not necessarily identically distributed.
 - Observations may come from different distributions (typically same class of distributions but different parameters).

Some goals of time series analysis or modeling

- Find the model that best describes the data and captures its underlying structure, and use it to identify key patterns, such as trends and recurring behaviors.
 - Forecast future values.
 - Identify and access variability in forecasts.

Classes of time series models to be covered

- We will cover two basic types of models:
 - Regression-based models that use the time components as explanatory variables to predict y .
 - Predict the response variable based on temporal patterns
 - Useful for capturing systematic patterns, such as trends and seasonality
 - Models that use past values ($y_{i-1}, y_{i-2}, \dots$) and past prediction errors ($e_{i-1}, e_{i-2}, \dots$) to model current values.
 - Capture temporal dependence in the data
 - Enable accurate forecasting by leveraging historical patterns
 - Commonly applied in econometrics, finance, atmospheric sciences, etc.

Types of time series

- Random walks: predominantly noise (e.g., stock returns)
 - No discernible patterns.
 - Any observed patterns are due to noise.

$$v_t = v_{t-1} + \epsilon_t, \quad \text{where } \epsilon \sim \mathcal{N}(\mu, \sigma)$$

- Full or partial combination of trend, seasonality, and noise (e.g., CO₂ levels in the atmosphere)

$$v_t = f(t) + s(t) + \epsilon_t$$

where f is a trend function and s is a seasonality function.

```
### Types of Time series: A Random Walk
```

```
v = np.zeros(1000)
```

```
mu = 0
```

```
sigma = 1
```

} $\epsilon \sim \mathcal{N}(0, 1)$

```
for t in range(1000):
```

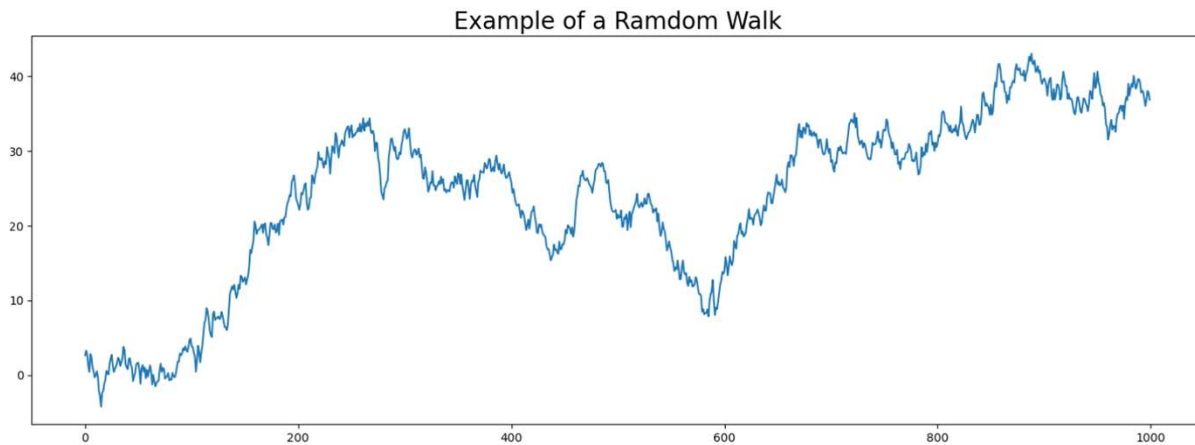
```
    v[t] = v[t-1] + np.random.normal(mu, sigma)
```

```
v[0:10]
```

```
array([ 2.64577585,  3.28492162,  2.7315797 ,  1.26172542,  0.43551446,  
        2.8244224 ,  2.37657022,  1.02680864,  0.44375362, -0.28964162])
```

```
plt.figure(figsize=(18, 6))

plt.plot(np.arange(1000), v)
plt.title("Example of a Random Walk", fontsize=20);
```



Types of Time series: Combination of trend, seasonality, and noise

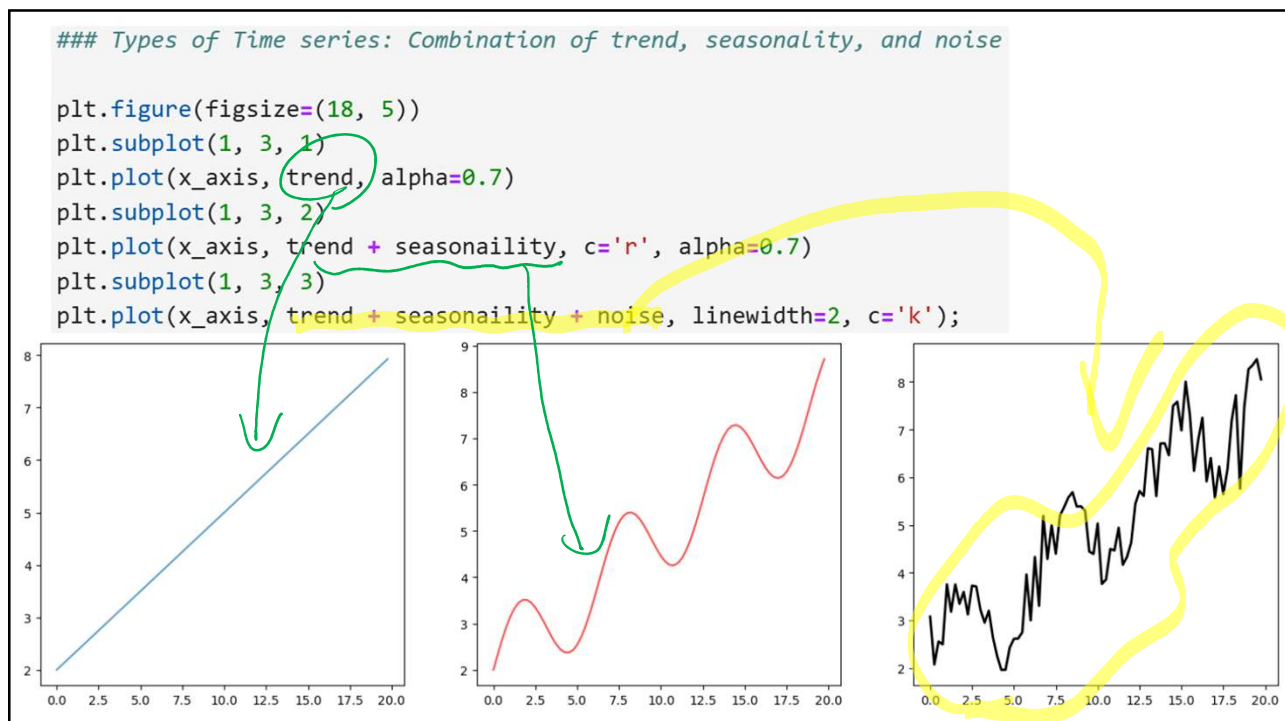
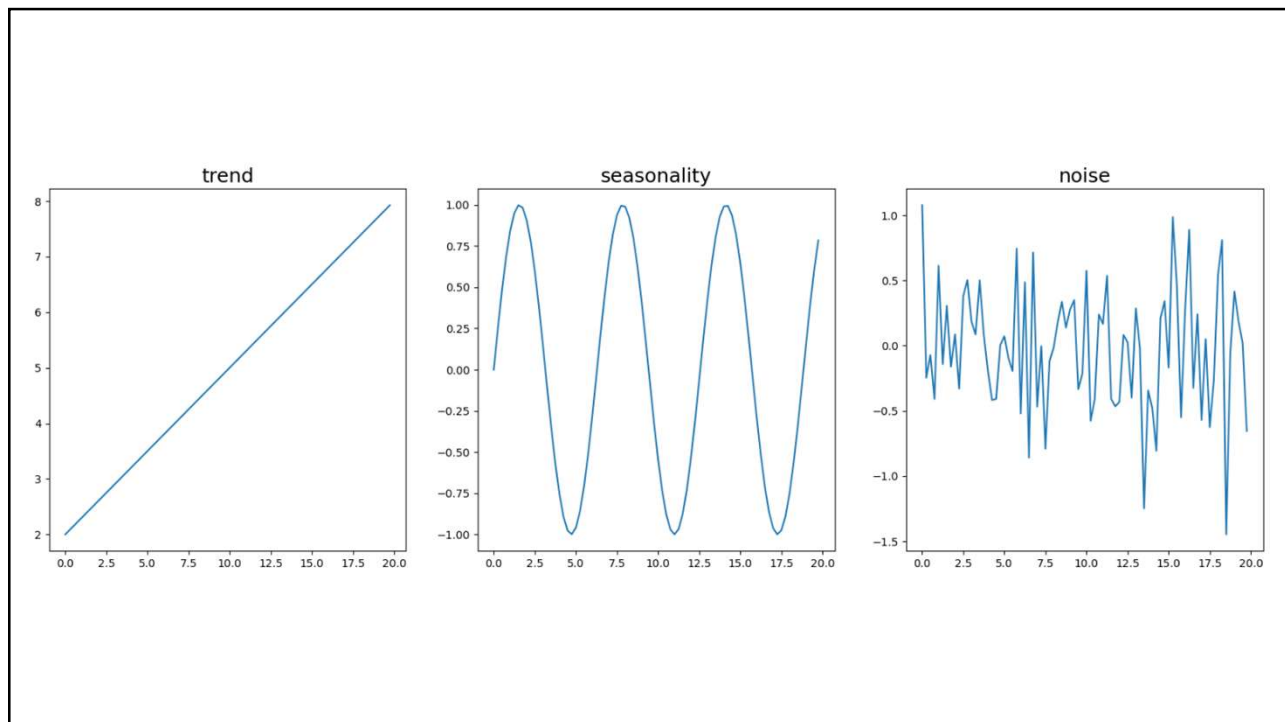
```
x_axis = np.arange(0, 20, 0.25)
```

```
trend = np.zeros(len(x_axis))
seasonality = np.zeros(len(x_axis))
noise = np.zeros(len(x_axis))
```

```
a = 2
b = 0.3
```

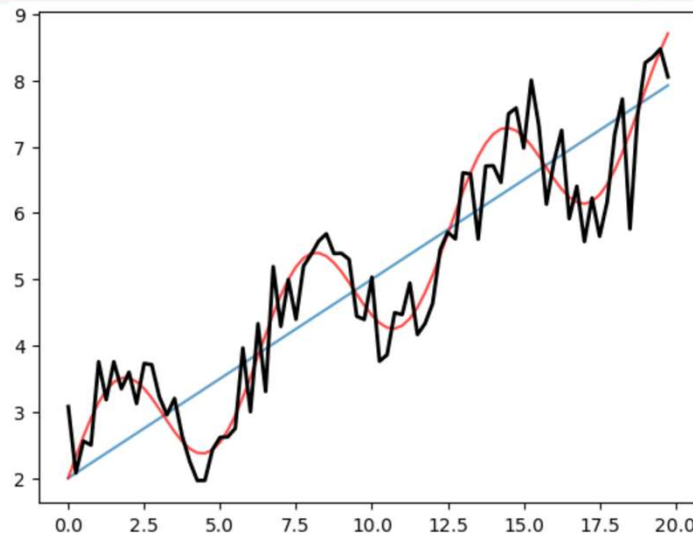
```
trend = a + b * x_axis
seasonality = np.sin(x_axis)
noise = np.random.normal(0, 0.5, len(x_axis))
```

$\nwarrow 2 + 0.3x$
 $\nwarrow \sin x$
 $\nwarrow \mathcal{N}(0, 0.5)$



```
### Types of Time series: Combination of trend, seasonality, and noise
```

```
plt.plot(x_axis, trend, alpha=0.7)  
plt.plot(x_axis, trend + seasonality, c='r', alpha=0.7)  
plt.plot(x_axis, trend + seasonality + noise, linewidth=2, c='k');
```



Time series example: Mauna Loa dataset

- Looking at CO₂ level values in the atmosphere as measured at Mauna Loa on the Big Island.
- The data was recorded monthly between January 1959 and December 1990.
- We model this dataset to highlight the three different components of time series.

32 years

Time Series is composed of:

Trend + Seasonality + Stochastic Fluctuations

```
co2_data = pd.read_csv("data/carbon_dioxide.txt", names=["co2_val"])
co2_data
```

	co2_val
0	315.42
1	316.32
2	316.49
3	317.56
4	318.13
...	...
379	352.68
380	350.72
381	350.92
382	352.55
383	353.91

384 rows × 1 columns

32 years × 12 = 384 months

Pandas and time series

- The pandas library makes it easy to work with time series.
- Extensive functionality to align multiple time series, subsample, impute missing values, etc.
- We will cover some functionality, but refer to Wes McKinney's book for more thorough coverage.
 - Chapter is available online (link in the course syllabus).
- Documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/timeseries.html.

```
# Generates ranges as timestamps, which are more appropriate for time series data
# Index is of type `pandas Timestamps`
```

```
dateIndex = pd.date_range('1/1/1959', periods=len(co2_data), freq="MS")
dateIndex
```

```
DatetimeIndex(['1959-01-01', '1959-02-01', '1959-03-01', '1959-04-01',
               '1959-05-01', '1959-06-01', '1959-07-01', '1959-08-01',
               '1959-09-01', '1959-10-01',
               ...
               '1990-03-01', '1990-04-01', '1990-05-01', '1990-06-01',
               '1990-07-01', '1990-08-01', '1990-09-01', '1990-10-01',
               '1990-11-01', '1990-12-01'],
              dtype='datetime64[ns]', length=384, freq='MS')
```

```
print(co2_data.index)
```

```
RangeIndex(start=0, stop=384, step=1)
```

```
# datetime64
```

```
co2_data.index = dateIndex
```

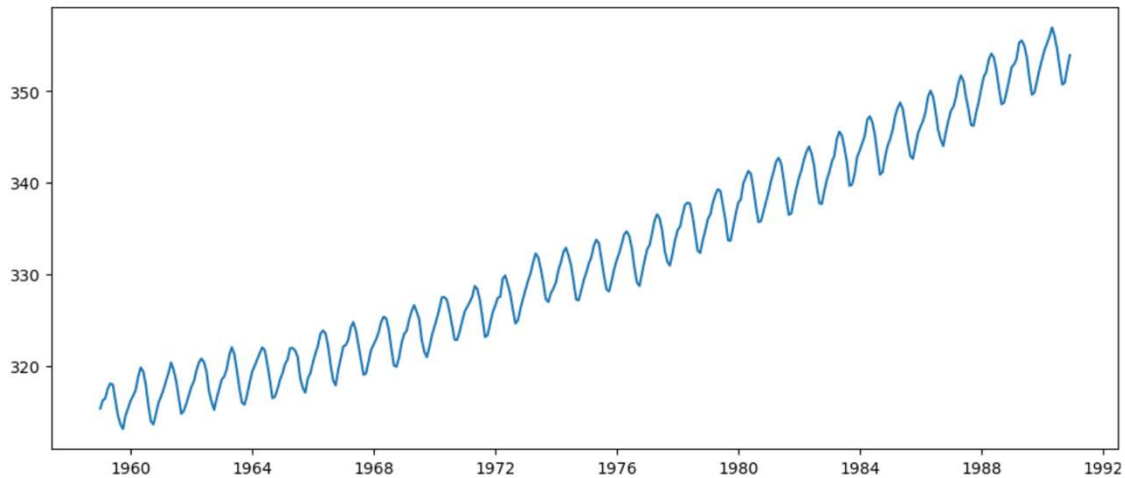
```
print(co2_data.index)
```

```
DatetimeIndex(['1959-01-01', '1959-02-01', '1959-03-01', '1959-04-01',
               '1959-05-01', '1959-06-01', '1959-07-01', '1959-08-01',
               '1959-09-01', '1959-10-01',
               ...
               '1990-03-01', '1990-04-01', '1990-05-01', '1990-06-01',
               '1990-07-01', '1990-08-01', '1990-09-01', '1990-10-01',
               '1990-11-01', '1990-12-01'],
              dtype='datetime64[ns]', length=384, freq='MS')
```

```
co2_data.head()
```

	co2_val
1959-01-01	315.42
1959-02-01	316.32
1959-03-01	316.49
1959-04-01	317.56
1959-05-01	318.13

```
plt.figure(figsize=(12, 5))  
plt.plot(co2_data['co2_val']);
```



Indexing datetime

- Values of DataFrame with a datetime64 index can be accessed using a partial string.
 - year or year-month or year-month-day, etc.
- This includes using partial strings in ranges (slicing).
 - The end index is included in the results.

```
co2_data.loc['1960']
```

y
co2_val

1960-01-01	316.27
1960-02-01	316.81
1960-03-01	317.42
1960-04-01	318.87
1960-05-01	319.87
1960-06-01	319.43
1960-07-01	318.01
1960-08-01	315.75
1960-09-01	314.00
1960-10-01	313.68
1960-11-01	314.84
1960-12-01	316.03

```
co2_data.loc['1960-4']
```

y-m
co2_val

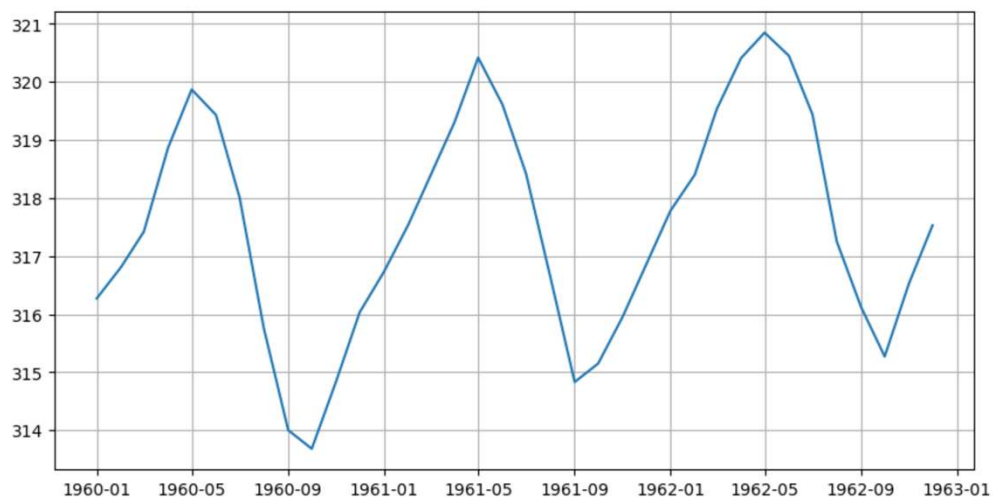
1960-04-01	318.87
-------------------	--------

```
co2_data.loc['1960-9':'1961-3']
```

co2_val

1960-09-01	314.00
1960-10-01	313.68
1960-11-01	314.84
1960-12-01	316.03
1961-01-01	316.73
1961-02-01	317.54
1961-03-01	318.38

```
plt.figure(figsize=(10, 5))  
plt.plot(co2_data.loc['1960':'1962'])  
plt.grid()
```



Accessing subsets of the datetime object

- Since the index is a timestamp (i.e., specific instant in time), we can easily access subsets of the data based on that timestamp.
 - Extract components such as day, day of the week, month, month_name, etc.
 - Enables filtering and grouping by calendar attributes
 - Supports analysis at different temporal resolutions
 - Useful for detecting seasonality and creating features for modeling

```
co2_data.index.day ← day of the month (1 ~ 31)
```

```
Index([1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
      ...,
      1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
      dtype='int32', length=384)
```

`co2_data.index.dayofweek` 0 ~ Monday
6 ~ Sunday

```
Index([3, 6, 6, 2, 4, 0, 2, 5, 1, 3,
      ...
      3, 6, 1, 4, 6, 2, 5, 0, 3, 5],
      dtype='int32', length=384)
```

```
# Format code in the datetime docs:
# https://docs.python.org/3/library/datetime.html#format-codes
# Formatting playground on https://www.strfti.me/

co2_data.index.strftime('%A') # full weekday name

Index(['Thursday', 'Sunday', 'Sunday', 'Wednesday', 'Friday', 'Monday',
      'Wednesday', 'Saturday', 'Tuesday', 'Thursday',
      ...,
      'Thursday', 'Sunday', 'Tuesday', 'Friday', 'Sunday', 'Wednesday',
      'Saturday', 'Monday', 'Thursday', 'Saturday'],
      dtype='object', length=384)

co2_data.index.strftime('%b') # abbreviated month name

Index(['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct',
      ...,
      'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'],
      dtype='object', length=384)

co2_data.index[:3].strftime('%a, %B %d, %Y')

Index(['Thu, January 01, 1959', 'Sun, February 01, 1959',
      'Sun, March 01, 1959'],
      dtype='object')
```

```
co2_data[co2_data.index.year.isin([1960, 1970])]
```

co2_val			
1960-01-01	316.27	1970-01-01	324.34
1960-02-01	316.81	1970-02-01	325.30
1960-03-01	317.42	1970-03-01	326.29
1960-04-01	318.87	1970-04-01	327.54
1960-05-01	319.87	1970-05-01	327.54
1960-06-01	319.43	1970-06-01	327.21
1960-07-01	318.01	1970-07-01	325.98
1960-08-01	315.75	1970-08-01	324.42
1960-09-01	314.00	1970-09-01	322.91
1960-10-01	313.68	1970-10-01	322.90
1960-11-01	314.84	1970-11-01	323.85
1960-12-01	316.03	1970-12-01	324.96

```
sample_4years = co2_data[co2_data.index.year.isin([1960, 1970, 1980, 1990])]
display(sample_4years.head())
display(sample_4years.tail())
```

	co2_val
--	---------

1960-01-01	316.27
------------	--------

1960-02-01	316.81
------------	--------

1960-03-01	317.42
------------	--------

1960-04-01	318.87
------------	--------

1960-05-01	319.87
------------	--------

```
print(sample_4years.index.year)
print("-----")
print(len(sample_4years.index.year))
```

	co2_val
--	---------

1990-08-01	352.68
------------	--------

1990-09-01	350.72
------------	--------

1990-10-01	350.92
------------	--------

1990-11-01	352.55
------------	--------

1990-12-01	353.91
------------	--------

```
Index([1960, 1960, 1960, 1960, 1960, 1960, 1960, 1960, 1960, 1960, 1960, 1960, 1960,
       1970, 1970, 1970, 1970, 1970, 1970, 1970, 1970, 1970, 1970, 1970, 1970, 1970,
       1980, 1980, 1980, 1980, 1980, 1980, 1980, 1980, 1980, 1980, 1980, 1980, 1980,
       1990, 1990, 1990, 1990, 1990, 1990, 1990, 1990, 1990, 1990, 1990, 1990, 1990],
      dtype='int32')
-----
48
```

```
sample_4years.loc['1960']
```

	co2_val
--	---------

1960-01-01	316.27
------------	--------

1960-02-01	316.81
------------	--------

1960-03-01	317.42
------------	--------

1960-04-01	318.87
------------	--------

1960-05-01	319.87
------------	--------

1960-06-01	319.43
------------	--------

1960-07-01	318.01
------------	--------

1960-08-01	315.75
------------	--------

1960-09-01	314.00
------------	--------

1960-10-01	313.68
------------	--------

1960-11-01	314.84
------------	--------

1960-12-01	316.03
------------	--------

Modeling the trend and seasonality

1. Exploring the trend and seasonality
2. Forming a hypothesis about the components
3. Writing and testing a model to account for the observed trend and seasonality
 - Naturally, we cannot model the residual noise.

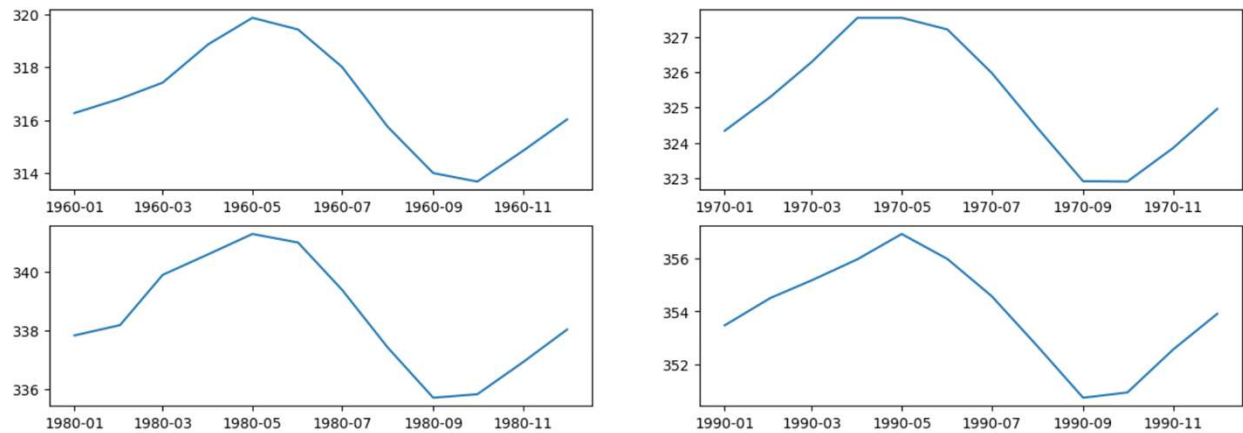
1. Exploring the data

- Data exploration is a crucial step in data analysis, especially for time series data.
- Some key questions to consider:
 - Does the data exhibit seasonality?
 - Where are the major lows and highs within the data and what are the values?
 - Is the observed seasonal pattern stable across different time periods?
 - How much variability exists over time?
- Essentially, this stage is exploratory in nature.
 - Any technique that enables the identification of trends and seasonality is appropriate.

```
plt.figure(figsize=(15, 5))

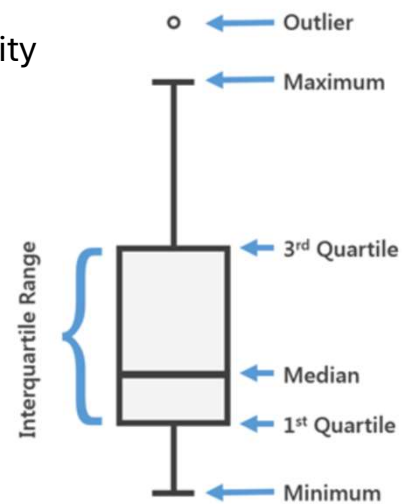
years = [1960, 1970, 1980, 1990]

for i, year in enumerate(years):
    plt.subplot(2, 2, i+1)
    plt.plot(sample_4years.loc[str(year)]['co2_val'])
```



Examine seasonal trend in the entire data

- We will use a boxplot to explore seasonality

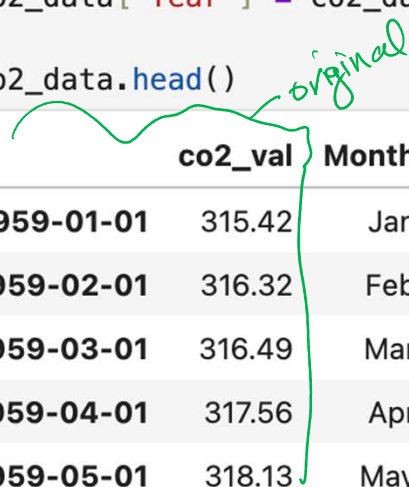


```
# Examine seasonal trend in the data
```

```
co2_data['Month'] = co2_data.index.strftime('%b')
```

```
co2_data['Year'] = co2_data.index.year
```

```
co2_data.head()
```



	co2_val	Month	Year
1959-01-01	315.42	Jan	1959
1959-02-01	316.32	Feb	1959
1959-03-01	316.49	Mar	1959
1959-04-01	317.56	Apr	1959
1959-05-01	318.13	May	1959

```
co2_data_piv = co2_data.pivot(index='Year', columns='Month', values='co2_val')
```

```
co2_data_piv.head()
```

Month	Apr	Aug	Dec	Feb	Jan	Jul	Jun	Mar	May	Nov	Oct	Sep
Year												
1959	317.56	314.66	315.43	316.32	315.42	316.39	318.00	316.49	318.13	314.66	313.18	313.68
1960	318.87	315.75	316.03	316.81	316.27	318.01	319.43	317.42	319.87	314.84	313.68	314.00
1961	319.31	316.64	316.85	317.54	316.73	318.42	319.61	318.38	320.42	315.95	315.15	314.83
1962	320.41	317.25	317.53	318.40	317.78	319.44	320.45	319.53	320.85	316.53	315.27	316.12
1963	321.22	317.61	318.20	318.92	318.58	319.58	321.31	319.70	322.08	316.91	315.83	316.05

32x12 table = 384 val.

```
# We reindex the data to have the months in order
```

```
orderedMonths = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"]
```

```
co2_data_piv = co2_data_piv.reindex(columns=orderedMonths)  
co2_data_piv.head()
```

Month	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
Year												
1959	315.42	316.32	316.49	317.56	318.13	318.00	316.39	314.66	313.68	313.18	314.66	315.43
1960	316.27	316.81	317.42	318.87	319.87	319.43	318.01	315.75	314.00	313.68	314.84	316.03
1961	316.73	317.54	318.38	319.31	320.42	319.61	318.42	316.64	314.83	315.15	315.95	316.85
1962	317.78	318.40	319.53	320.41	320.85	320.45	319.44	317.25	316.12	315.27	316.53	317.53
1963	318.58	318.92	319.70	321.22	322.08	321.31	319.58	317.61	316.05	315.83	316.91	318.20

```
co2_data_piv.shape
```

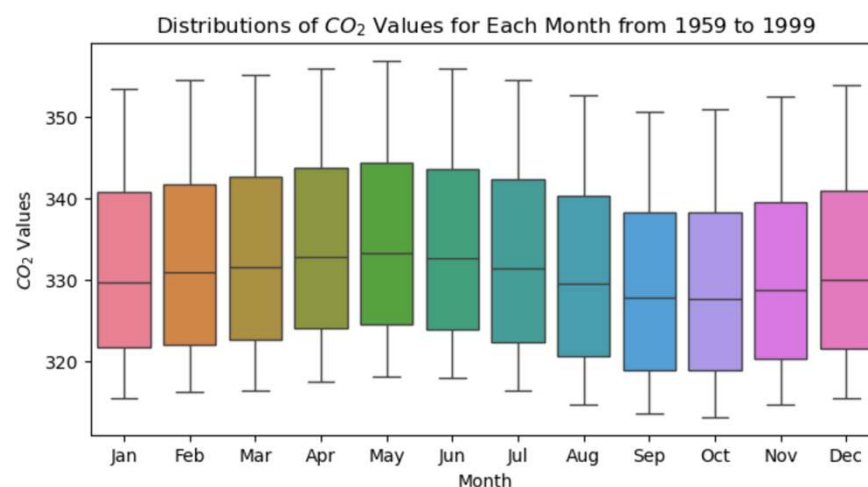
```
(32, 12)
```

```
plt.figure(figsize=(8, 4))
```

```
sns.boxplot(data=co2_data_piv)
```

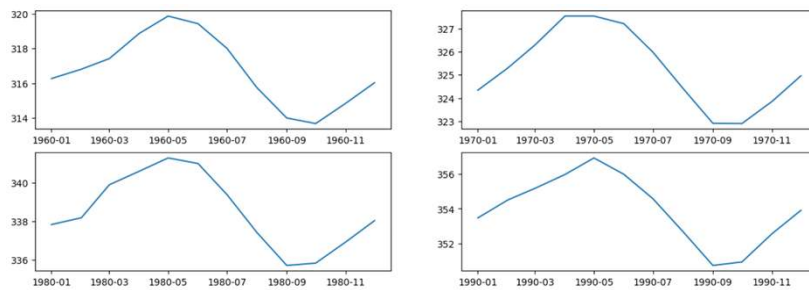
```
plt.title('Distributions of $CO_2$ Values for Each Month from 1959 to 1999')
```

```
plt.ylabel('$CO_2$ Values');
```



2. Forming a hypothesis

- The seasonality hypothesis is supported by the boxplots.
- Earlier, we only explored four years-worth of data, but both representations indicate the presence of a consistent seasonality signal in the data.



The seasonality looks sinusoidal with a period of one year.